Rapport de TP : Trino, labs 2 et 3

Gaëtan Zinck Dias Blanco | Namespace : user-g-zinck--diasblanco-ece | Exécuté le 09/11/2026 à 11:24 | Toutes les sorties ci-dessous sont celles de cette exécution.

Lab 2 : Silver to gold avec Trino

Environnement

Connexion au coordinateur Trino depuis Python avec le client `trino`. Le paramètre catalog="lakehouse" définit le catalogue par défaut de la session.

Commande

```
conn = trino.dbapi.connect(host="trino", port=8080, user=os.environ["TRINO_USER"],
                           http_scheme="http", catalog="lakehouse")
```

Commande

```
sql("""
SELECT version()
""")
```

Sortie

```
  _col0
0   480
```

Effet du catalogue par défaut : les noms non qualifiés sont résolus dans `lakehouse`.

Commande

```
sql("""
SHOW SCHEMAS
""")
```

Sortie

```
              Schema
0  information_schema
1              silver
2              system
```

Commande

```
sql("""
SELECT count(*) AS orders FROM silver.orders
""")
```

Sortie

```
   orders
0    2829
```

Question : Que change le paramètre catalog= de connect() dans les instructions suivantes ?

Il fixe le catalogue par défaut de la session. Un nom à deux niveaux comme `silver.orders` est résolu en `lakehouse.silver.orders`. La requête non qualifiée ci-dessus renvoie 2829 lignes, exactement comme la forme complète. De même, `SHOW SCHEMAS` sans FROM liste les schémas de `lakehouse`. Les noms complets `catalogue.schéma.table` ne changent pas. Le schéma reste obligatoire, car aucun `schema=` n'est passé à connect().

Catalogues, schémas et tables

Commande

```
sql("""
SHOW CATALOGS
""")
```

Sortie

```
     Catalog
0  lakehouse
1     system
2      tpcds
3       tpch
```

Commande

```
sql("""
SHOW SCHEMAS FROM lakehouse
""")
```

Sortie

```
              Schema
0  information_schema
1              silver
2              system
```

Commande

```
sql("""
SHOW TABLES FROM lakehouse.silver
""")
```

Sortie

```
    Table
0  orders
1   users
```

Commande

```
sql("""
DESCRIBE lakehouse.silver.orders
""")
```

Sortie

```
       Column                       Type Extra Comment
0    order_id                    varchar
1     user_id                    varchar
2  ordered_at  timestamp(6) with time zone
3    quantity                    integer
4     product                    varchar
```

Commande

```
sql("""
DESCRIBE lakehouse.silver.users
""")
```

Sortie

```
      Column     Type Extra Comment
0    user_id  varchar
1   username  varchar
2       name  varchar
3        sex  varchar
4      email  varchar
5  birthdate     date
6     street  varchar
7       city  varchar
8      state  varchar
9   zip_code  varchar
```

Question : Quel type Trino porte ordered_at ? Garde-t-il un fuseau horaire ?

Trino rapporte `timestamp(6) with time zone`. Spark a écrit un TimestampType, stocké en Iceberg comme `timestamptz`, c'est-à-dire un instant normalisé en UTC. Trino l'expose donc « with time zone » et affiche les valeurs en UTC (+00:00). L'instant est conservé. Le décalage d'origine de la source n'est pas conservé : toutes les valeurs sont ramenées à UTC.

Question : Spark a déclaré order_id en string faute de type UUID. Que rapporte Trino ?

Trino rapporte `varchar`. Le fichier de métadonnées Iceberg déclare une colonne `string`, et Trino ne peut pas deviner qu'elle contient des UUID. Il la traite donc comme du texte : aucune validation de format, une comparaison et un stockage plus coûteux qu'un UUID binaire de 16 octets. Pour obtenir un vrai type `uuid`, il faudrait déclarer explicitement la colonne en `uuid` dans Iceberg à la création de la table.

Interroger la couche silver

Commande

```
sql("""
SELECT count(*) AS orders FROM lakehouse.silver.orders
""")
```

Sortie

```
   orders
0    2829
```

Commande

```
sql("""
SELECT count(*) AS users FROM lakehouse.silver.users
""")
```

Sortie

```
   users
0     50
```

Commande

```
sql("""
SELECT state, count(*) AS users, count_if(city IS NULL) AS missing_city
FROM lakehouse.silver.users
GROUP BY state
ORDER BY users DESC
LIMIT 10
""")
```

Sortie

```
  state  users  missing_city
0    FM      3             0
1    AE      3             3
2    VI      2             0
3    AP      2             2
4    NM      2             0
5    VT      2             0
6    TX      2             0
7    PA      2             0
8    HI      2             0
9    NJ      2             0
```

Commande

```
sql("""
SELECT count(*) AS orphan_orders
FROM lakehouse.silver.orders AS o
LEFT JOIN lakehouse.silver.users AS u ON u.user_id = o.user_id
WHERE u.user_id IS NULL
""")
```

Sortie

```
   orphan_orders
0              0
```

Question : Expliquer la sortie des requêtes exécutées.

La couche silver contient 2829 commandes et 50 utilisateurs. Le texte du lab annonce 5000 commandes, mais le générateur (graine 42) produit 2829 commandes pour 50 utilisateurs, chiffre confirmé par le lab DuckDB et par la taille du fichier bronze (331 568 octets). Le dédoublonnage de Spark n'a donc rien retiré. La répartition par état montre que les « états » sans ville (AE (3/3), AP (2/2)) sont des codes militaires américains (AA, AE, AP) : les adresses de type « Unit ... Box ... DPO AP 09617 »

n'ont pas de ville, et la regex laisse alors `city` à NULL, ce qui est le comportement voulu. La requête d'orphelins renvoie 0 : chaque commande a un utilisateur correspondant, donc une jointure interne ne perdra aucune ligne.

Question : Quel test du lab dbt la requête d'orphelins remplace-t-elle ? Que faut-il écrire pour l'exécuter à chaque rafraîchissement ?

Elle remplace le test `relationships` de dbt (stg_orders.user_id doit exister dans stg_users.user_id), qui vérifie l'intégrité référentielle. Pour l'exécuter à chaque rafraîchissement, il faut en faire un contrôle bloquant dans le pipeline : soit le test `relationships` déclaré dans le schema.yml d'un projet dbt-trino et lancé par `dbt build`, soit une étape du pipeline qui exécute la requête et échoue si le résultat est supérieur à 0 (par exemple un `assert` Python, ou une tâche de l'orchestrateur), placée avant la construction de la couche gold.

Question : Trino a répondu sans job à soumettre ni session à créer. Qu'est-ce que cela dit sur le moment où choisir Trino plutôt que Spark ?

Trino est un service MPP qui tourne en permanence : le coordinateur et les workers sont déjà démarrés, avec les catalogues configurés côté serveur. Une requête SQL s'exécute donc en quelques centaines de millisecondes, sans démarrer de JVM ni télécharger de dépendances. Trino est le bon choix pour les requêtes interactives, ad hoc et BI (latence faible, nombreux utilisateurs simultanés). Spark convient mieux aux traitements batch lourds et longs, aux transformations programmatiques (Python, ML) et aux jobs qui doivent survivre à la perte d'un nœud.

Métadonnées Iceberg

Commande

```
sql("""
SELECT snapshot_id, committed_at, operation,
       element_at(summary, 'engine-name') AS engine
FROM lakehouse.silver."orders$snapshots"
ORDER BY committed_at
""")
```

Sortie

```
           snapshot_id                committed_at  operation engine
0  5494453151731800034 2026-11-09 10:53:23.568+00:00  overwrite  spark
```

Commande

```
sql("""
SELECT file_path, file_format, record_count, file_size_in_bytes
FROM lakehouse.silver."orders$files"
""")
```

Sortie

```
file_path file_format  record_count  file_size_in_bytes
0  s3a://user-g-zinck--diasblanco-ece/warehouse/silver/orders/data/00000-6-47527fed-8bf3-4144-8b7c-a059d8e98e79-0-
00001.parquet     PARQUET          2829               84055
```

Commande

```
sql("""
SELECT * FROM lakehouse.silver."orders$history"
""")
```

Sortie

```
                made_current_at          snapshot_id parent_id  is_current_ancestor
0 2026-11-09 10:53:23.568+00:00  5494453151731800034      None                 True
```

Commande

```
aws s3 --profile default ls "s3://user-g-zinck--diasblanco-ece/warehouse/silver/orders/" --recursive
```

Sortie

```
2026-11-09 10:53:23      84055 warehouse/silver/orders/data/00000-6-47527fed-8bf3-4144-8b7c-a059d8e98e79-0-00001.
parquet
2026-11-09 10:53:23       7358 warehouse/silver/orders/metadata/8a202a23-cd91-4df4-bc0a-66a78ce933ce-m0.avro
2026-11-09 10:53:23       4463 warehouse/silver/orders/metadata/snap-5494453151731800034-1-8a202a23-cd91-4df4-bc0a-
66a78ce933ce.avro
2026-11-09 10:53:23       1955 warehouse/silver/orders/metadata/v1.metadata.json
2026-11-09 10:53:24          1 warehouse/silver/orders/metadata/version-hint.text
```

Question : Combien de snapshots silver.orders a-t-elle ? Quel moteur les a écrits ?

1 snapshot(s), opération(s) : overwrite. La colonne `engine`, lue dans le résumé du snapshot, vaut : spark. Ils ont donc été écrits par Spark lors du lab Spark (writeTo...createOrReplace()). L'enregistrement dans Polaris (register_table) ne crée aucun snapshot : il ajoute seulement un pointeur vers le fichier de métadonnées existant. Le lab en montre 3 parce que le lab Iceberg (module 08) ajoute deux snapshots « append ».

Question : Lister les fichiers avec aws s3 ls et comparer avec $files. Lequel n'a pas besoin d'accéder au bucket (en listing), et pourquoi ?

`$files` renvoie 1 fichier(s) de données. `aws s3 ls` liste 5 objets, dont 1 sous data/. Il montre aussi les fichiers de métadonnées (metadata.json, manifestes avro, version-hint) et, après des réécritures, des fichiers de données qui ne sont plus référencés. `$files` n'a pas besoin de lister le bucket : Trino demande à Polaris l'emplacement du metadata.json courant, puis lit la manifest list et les manifestes, qui contiennent la liste exacte des fichiers du snapshot courant, avec leur nombre de lignes et leur taille. C'est le principe d'Iceberg : la liste des fichiers est dans les métadonnées, pas dans un LIST S3 (coûteux et éventuellement incohérent sur un gros bucket).

Plan de requête et exécution

Commande

```
print(sql("""
EXPLAIN (TYPE DISTRIBUTED)
SELECT u.state, count(*) AS orders
FROM lakehouse.silver.orders AS o
JOIN lakehouse.silver.users  AS u ON u.user_id = o.user_id
GROUP BY u.state
""").iloc[0, 0])
```

Sortie

```
Trino version: 480
Fragment 0 [HASH]
    Output layout: [state, count]
    Output partitioning: SINGLE []
    Output[columnNames = [state, orders]]
    │   Layout: [state:varchar, count:bigint]
    │   Estimates: {rows: ? (?), cpu: 0, memory: 0B, network: 0B}
    │   orders := count
    └─ Aggregate[type = FINAL, keys = [state]]
       │   Layout: [state:varchar, count:bigint]
       │   Estimates: {rows: ? (?), cpu: ?, memory: ?, network: 0B}
       │   count := count(count_4)
       └─ LocalExchange[partitioning = HASH, arguments = [state::varchar]]
          │   Layout: [state:varchar, count_4:bigint]
          │   Estimates: {rows: ? (?), cpu: ?, memory: 0B, network: 0B}
          └─ RemoteSource[sourceFragmentIds = [1]]
                Layout: [state:varchar, count_4:bigint]

Fragment 1 [SOURCE]
    Output layout: [state, count_4]
    Output partitioning: HASH [state]
    Aggregate[type = PARTIAL, keys = [state]]
    │   Layout: [state:varchar, count_4:bigint]
    │   count_4 := count(*)
    └─ InnerJoin[criteria = (user_id = user_id_0), distribution = REPLICATED]
       │   Layout: [state:varchar]
       │   Estimates: {rows: ? (?), cpu: ?, memory: 4.02kB, network: 0B}
       │   Distribution: REPLICATED
       │   dynamicFilterAssignments = {user_id_0 -> #df_314}
       ├─ ScanFilter[table = lakehouse:silver.orders$data@5494453151731800034, dynamicFilters = {user_id = #df_314}]
       │      Layout: [user_id:varchar]
       │      Estimates: {rows: 2829 (22.36kB), cpu: 22.36k, memory: 0B, network: 0B}/{rows: 2829 (22.36kB), cpu: 22.
36k, memory: 0B, network: 0B}
       │      user_id := 2:user_id:varchar
       └─ LocalExchange[partitioning = SINGLE]
          │   Layout: [user_id_0:varchar, state:varchar]
          │   Estimates: {rows: 50 (4.02kB), cpu: 0, memory: 0B, network: 0B}
          └─ RemoteSource[sourceFragmentIds = [2]]
                Layout: [user_id_0:varchar, state:varchar]

Fragment 2 [SOURCE]
    Output layout: [user_id_0, state]
    Output partitioning: BROADCAST []
    TableScan[table = lakehouse:silver.users$data@6152640127490600713]
        Layout: [user_id_0:varchar, state:varchar]
        Estimates: {rows: 50 (4.02kB), cpu: 4.02k, memory: 0B, network: 0B}
        state := 9:state:varchar
        user_id_0 := 1:user_id:varchar
```

Lecture du plan : 3 fragments (un par stage), 1 opérateurs TableScan (un par table), 2 RemoteSource (échanges REMOTE entre fragments), distribution de la jointure : REPLICATED. Avec 50 utilisateurs contre 2829 commandes, l'optimiseur à base de coûts diffuse la petite table (REPLICATED, broadcast) au lieu de repartitionner les deux côtés sur user_id (PARTITIONED).

Commande

```
print(sql("""
EXPLAIN ANALYZE
SELECT u.state, count(*) AS orders
FROM lakehouse.silver.orders AS o
JOIN lakehouse.silver.users  AS u ON u.user_id = o.user_id
GROUP BY u.state
""").iloc[0, 0])
```

Sortie

```
Trino version: 480
Queued: 374.94us, Analysis: 926.35ms, Planning: 488.90ms, Execution: 2.67s, Finishing: 0.00ns
Fragment 1 [HASH]
    CPU: 5.37ms, Scheduled: 43.82ms, Blocked 7.16s (Input: 1.76s, Output: 0.00ns), Input: 37 rows (592B); per task:
avg.: 37.00 std.dev.: 0.00, Output: 37 rows (592B)
    Peak Memory: 840B, Tasks count: 1; per task: max: 840B
    Output layout: [state, count]
    Output partitioning: SINGLE []
    Aggregate[type = FINAL, keys = [state]]
    │   Layout: [state:varchar, count:bigint]
    │   Estimates: {rows: ? (?), cpu: ?, memory: ?, network: 0B}
    │   CPU: 3.00ms (1.54%), Scheduled: 3.00ms (0.42%), Blocked: 0.00ns (0.00%), Output: 37 rows (592B)
    │   Input avg.: 18.50 rows, Input std.dev.: 2.70%
    │   count := count(count_4)
    └─ LocalExchange[partitioning = HASH, arguments = [state::varchar]]
       │   Layout: [state:varchar, count_4:bigint]
       │   Estimates: {rows: ? (?), cpu: ?, memory: 0B, network: 0B}
       │   CPU: 0.00ns (0.00%), Scheduled: 38.00ms (5.34%), Blocked: 3.53s (58.63%), Output: 37 rows (592B)
       │   Input avg.: 18.50 rows, Input std.dev.: 100.00%
       └─ RemoteSource[sourceFragmentIds = [2]]
              Layout: [state:varchar, count_4:bigint]
              CPU: 0.00ns (0.00%), Scheduled: 1.00ms (0.14%), Blocked: 1.76s (29.23%), Output: 37 rows (592B)
              Input avg.: 18.50 rows, Input std.dev.: 100.00%

Fragment 2 [SOURCE]
    CPU: 173.02ms, Scheduled: 567.37ms, Blocked 1.69s (Input: 484.81ms, Output: 0.00ns), Input: 2879 rows (126.66kB);
per task: avg.: 2879.00 std.dev.: 0.00, Output: 37 rows (592B)
    Peak Memory: 128.42kB, Tasks count: 1; per task: max: 128.42kB
    Output layout: [state, count_4]
    Output partitioning: HASH [state]
    Aggregate[type = PARTIAL, keys = [state]]
    │   Layout: [state:varchar, count_4:bigint]
    │   CPU: 23.00ms (11.79%), Scheduled: 72.00ms (10.11%), Blocked: 0.00ns (0.00%), Output: 37 rows (592B)
    │   Input avg.: 2829.00 rows, Input std.dev.: 0.00%
    │   count_4 := count(*)
    └─ InnerJoin[criteria = (user_id = user_id_0), distribution = REPLICATED]
       │   Layout: [state:varchar]
       │   Estimates: {rows: ? (?), cpu: ?, memory: 4.02kB, network: 0B}
       │   CPU: 55.00ms (28.21%), Scheduled: 209.00ms (29.35%), Blocked: 0.00ns (0.00%), Output: 2829 rows (19.34kB)
       │   Left (probe) Input avg.: 2829.00 rows, Input std.dev.: 0.00%
       │   Right (build) Input avg.: 50.00 rows, Input std.dev.: 0.00%
       │   Distribution: REPLICATED
       │   dynamicFilterAssignments = {user_id_0 -> #df_373}
       ├─ ScanFilter[table = lakehouse:silver.orders$data@5494453151731800034, dynamicFilters = {user_id = #df_373}]
       │      Layout: [user_id:varchar]
       │      Estimates: {rows: 2829 (22.36kB), cpu: 22.36k, memory: 0B, network: 0B}/{rows: 2829 (22.36kB), cpu: 22.
36k, memory: 0B, network: 0B}
       │      CPU: 93.00ms (47.69%), Scheduled: 281.00ms (39.47%), Blocked: 0.00ns (0.00%), Output: 2829 rows (123.
33kB)
       │      Input avg.: 2829.00 rows, Input std.dev.: 0.00%
       │      user_id := 2:user_id:varchar
       │      Input: 2829 rows (124.32kB), Filtered: 0.00%, Physical input: 82.08kB, Physical input time: 67.36us,
Splits: 1, Splits generation wait time: 757.76ms
       │      Dynamic filters:
       │          - df_373, [ SortedRangeSet[type=varchar, ranges=50, {[09a0a0f6-73b0-4091-b634-c1694f76e388], ..., [
f94d6204-6808-493f-9fed-2c43e256a6dc]}] ], collection time=988.21ms
       └─ LocalExchange[partitioning = SINGLE]
          │   Layout: [user_id_0:varchar, state:varchar]
          │   Estimates: {rows: 50 (4.02kB), cpu: 0, memory: 0B, network: 0B}
          │   CPU: 0.00ns (0.00%), Scheduled: 3.00ms (0.42%), Blocked: 246.00ms (4.09%), Output: 50 rows (2.34kB)
          │   Input avg.: 25.00 rows, Input std.dev.: 100.00%
          └─ RemoteSource[sourceFragmentIds = [3]]
                 Layout: [user_id_0:varchar, state:varchar]
                 CPU: 1.00ms (0.51%), Scheduled: 1.00ms (0.14%), Blocked: 485.00ms (8.06%), Output: 50 rows (2.34kB)
                 Input avg.: 25.00 rows, Input std.dev.: 100.00%
```

```
Fragment 3 [SOURCE]
    CPU: 20.63ms, Scheduled: 103.73ms, Blocked 0.00ns (Input: 0.00ns, Output: 0.00ns), Input: 50 rows (2.54kB); per
task: avg.: 50.00 std.dev.: 0.00, Output: 50 rows (2.54kB)
    Peak Memory: 3.62kB, Tasks count: 1; per task: max: 3.62kB
    Output layout: [user_id_0, state]
    Output partitioning: BROADCAST []
    TableScan[table = lakehouse:silver.users$data@6152640127490600713]
        Layout: [user_id_0:varchar, state:varchar]
        Estimates: {rows: 50 (4.02kB), cpu: 4.02k, memory: 0B, network: 0B}
        CPU: 20.00ms (10.26%), Scheduled: 104.00ms (14.61%), Blocked: 0.00ns (0.00%), Output: 50 rows (2.54kB)
        Input avg.: 50.00 rows, Input std.dev.: 0.00%
        state := 9:state:varchar
        user_id_0 := 1:user_id:varchar
        Input: 50 rows (2.54kB), Physical input: 7.26kB, Physical input time: 30.48us, Splits: 1, Splits generation
wait time: 196.28ms
```

Commande

```
sql("""
SELECT query_id, state,
       date_diff('millisecond', created, coalesce("end", current_timestamp)) AS elapsed_ms,
       substr(query, 1, 60) AS query
FROM system.runtime.queries
ORDER BY created DESC
LIMIT 5
""")
```

Sortie

```
                     query_id     state  elapsed_ms                                                          query
0  20261109_112226_00038_7c7wa   RUNNING         -87   SELECT query_id, state,\n       date_diff('millisecond', crea
1  20261109_112222_00037_7c7wa  FINISHED        3711  EXPLAIN ANALYZE\nSELECT u.state, count(*) AS orders\nFROM lake
2  20261109_112221_00036_7c7wa  FINISHED        1267   EXPLAIN (TYPE DISTRIBUTED)\nSELECT u.state, count(*) AS order
3  20261109_112218_00035_7c7wa  FINISHED         566                SELECT * FROM lakehouse.silver."orders$history"
4  20261109_112217_00034_7c7wa  FINISHED         886    SELECT file_path, file_format, record_count, file_size_in_by
```

Interface web : le chart est déployé avec `ingress.enabled: false`, le coordinateur n'est donc pas exposé hors du cluster. L'interface /ui n'est accessible que par `kubectl port-forward svc/trino 8080:8080`, ce qui ne permet pas de l'ouvrir depuis un navigateur dans l'environnement Onyxia. La table system.runtime.queries ci-dessus donne le même historique.

Couche gold

Commande

```
sql("""
CREATE SCHEMA IF NOT EXISTS lakehouse.gold
""")
```

Sortie

```
(instruction exécutée, pas de résultat)
```

Commande

```
sql("""
SHOW SCHEMAS FROM lakehouse
""")
```

Sortie

```
              Schema
0               gold
1  information_schema
2             silver
3             system
```

Commandes par état et par mois

Commande

```
sql("""
CREATE TABLE lakehouse.gold.orders_by_state_month AS
SELECT
    u.state,
    date_trunc('month', o.ordered_at) AS month,
    count(*)                          AS orders,
    sum(o.quantity)                   AS units,
    count(DISTINCT o.user_id)         AS buyers
FROM lakehouse.silver.orders AS o
JOIN lakehouse.silver.users  AS u ON u.user_id = o.user_id
GROUP BY 1, 2
""")
```

Sortie

```
   rows
0    50
```

Commande

```
sql("""
SELECT * FROM lakehouse.gold.orders_by_state_month ORDER BY month, state LIMIT 10
""")
```

Sortie

```
  state                     month  orders  units  buyers
0    AL 2020-01-01 00:00:00+00:00      65    199       1
1    AP 2020-01-01 00:00:00+00:00      17     49       1
2    AS 2020-01-01 00:00:00+00:00      12     40       1
3    CO 2020-01-01 00:00:00+00:00      17     43       1
4    FM 2020-01-01 00:00:00+00:00      66    200       1
5    GA 2020-01-01 00:00:00+00:00       4     12       1
6    IN 2020-01-01 00:00:00+00:00      49    175       1
7    KY 2020-01-01 00:00:00+00:00      83    256       1
8    MH 2020-01-01 00:00:00+00:00      21     57       1
9    NE 2020-01-01 00:00:00+00:00      11     30       1
```

Produits les plus vendus

Commande

```
sql("""
CREATE TABLE lakehouse.gold.top_products AS
SELECT
    product,
    count(*)      AS orders,
    sum(quantity) AS units,
    rank() OVER (ORDER BY sum(quantity) DESC) AS units_rank
FROM lakehouse.silver.orders
GROUP BY product
""")
```

Sortie

```
   rows
0     6
```

Commande

```
sql("""
SELECT * FROM lakehouse.gold.top_products ORDER BY units_rank LIMIT 10
""")
```

Sortie

```
     product  orders  units  units_rank
0  croissant     491   1523           1
1      drink     484   1493           2
2    brioche     468   1424           3
3      bread     458   1420           4
4      donut     462   1390           5
5     cookie     466   1365           6
```

Activité de chaque utilisateur

Commande

```
sql("""
CREATE TABLE lakehouse.gold.user_activity AS
SELECT
    u.user_id,
    u.state,
    date_diff('year', u.birthdate, current_date) AS age,
    count(o.order_id)            AS orders,
    coalesce(sum(o.quantity), 0) AS units,
    min(o.ordered_at)            AS first_order_at,
    max(o.ordered_at)            AS last_order_at
FROM lakehouse.silver.users AS u
LEFT JOIN lakehouse.silver.orders AS o ON o.user_id = u.user_id
GROUP BY u.user_id, u.state, u.birthdate
""")
```

Sortie

```
   rows
0    50
```

Commande

```
sql("""
SELECT * FROM lakehouse.gold.user_activity ORDER BY orders DESC LIMIT 10
""")
```

Sortie

```
                               user_id state  age  orders  units                   first_order_at
last_order_at
0  9793b9b4-1374-4146-b2c5-bd89b70b3420    HI   65     100    289 2020-02-06 22:28:23.377915+00:00 2020-02-11 01:57:
32.773432+00:00
1  2051acef-097a-4e10-b6fe-bc0e7ecddbaf    VI    9     100    301 2020-03-13 06:21:16.892555+00:00 2020-03-17 09:02:
08.476647+00:00
2  8d605936-0380-4b70-8d03-c91e4f8d5238    WA   76      99    280 2020-03-06 12:42:24.432345+00:00 2020-03-10 14:46:
03.127489+00:00
3  52828d80-44b5-41f7-97ac-6aa8bb2488a3    IA   35      97    303 2020-02-11 02:16:02.488268+00:00 2020-02-15 02:18:
44.854831+00:00
4  72bb912d-7da6-4785-b63b-4dc3a559e463    NJ   74      97    317 2020-02-15 03:39:36.282921+00:00 2020-02-19 03:23:
18.610446+00:00
5  97fa7f04-8363-4007-9cae-9610c72c1fe3    AP   52      92    285 2020-03-27 11:09:51.257559+00:00 2020-03-31 06:01:
03.068375+00:00
6  0dde29a6-baa4-471a-9d24-67ac778eedb3    VI   85      92    285 2020-01-08 11:20:45.094193+00:00 2020-01-12 06:23:
02.514874+00:00
7  90164161-cfa7-41cd-a631-d00b26d794d3    CA   82      91    270 2020-02-22 14:30:47.467747+00:00 2020-02-26 08:13:
22.890319+00:00
8  832920b7-c143-4426-b72f-871a45ee432d    HI   51      91    272 2020-04-08 18:29:41.913897+00:00 2020-04-12 12:41:
22.197741+00:00
9  d045dd1c-6684-49e3-b1f8-343ea99f1318    UT   33      86    256 2020-03-02 22:15:57.890701+00:00 2020-03-06 11:03:
10.312652+00:00
```

Commande

```
sql("""
SHOW TABLES FROM lakehouse.gold
""")
```

Sortie

```
                  Table
0  orders_by_state_month
1           top_products
2          user_activity
```

Commande

```
sql("""
SELECT file_path, record_count, file_size_in_bytes
FROM lakehouse.gold."top_products$files"
""")
```

Sortie

```
                                              file_path  record_count  file_size_in_bytes
0  s3://user-g-zinck--diasblanco-ece/warehouse/gold/top_products-e576865a18e54cdcb82d09ceea44e629/data/
20261109_112240_00043_7c7wa-89c719f9-9f06-4f84-a31d-8954cc91c6f9.parquet             6                 760
```

Commande

```
sql("""
SELECT count_if(orders = 0) AS users_without_order FROM lakehouse.gold.user_activity
""")
```

Sortie

```
   users_without_order
0                    0
```

Question : Pourquoi user_activity utilise un LEFT JOIN et orders_by_state_month une jointure interne ? Laquelle dépend du résultat de la requête d'orphelins ?

`user_activity` décrit chaque utilisateur, y compris ceux qui n'ont jamais commandé : le LEFT JOIN depuis users les conserve, avec orders = 0 et units = 0 grâce à coalesce. Ici, 0 utilisateur(s) sans commande, car le générateur donne au moins une commande à chacun ; le LEFT JOIN protège le modèle pour des données futures. `orders_by_state_month` agrège des commandes par l'état de l'acheteur : une commande sans utilisateur n'a pas d'état, et la jointure interne la supprime silencieusement. C'est cette table qui dépend de la requête d'orphelins. Avec 0 orphelin(s), elle ne perd

rien, et la validation sum(units) = sum(silver.quantity) le confirme plus bas.

Question : current_date fait changer age chaque jour. Qu'est-ce que cela implique pour le rafraîchissement de la table ?

La table est figée au jour du CREATE TABLE AS SELECT : dès le lendemain, age peut être faux pour les utilisateurs dont c'est l'anniversaire. Le résultat dépend de la date d'exécution, donc il n'est pas reproductible (deux exécutions de la même requête sur les mêmes données donnent des tables différentes, et le time travel montre l'âge de l'époque). Il faut soit rafraîchir la table au moins une fois par jour, soit stocker `birthdate` et calculer l'âge au moment de la lecture, par exemple dans une vue.

Question : Où vivent les fichiers de données des tables gold, et qui a décidé de leur emplacement ?

Dans le bucket, sous `s3://user-g-zinck--diasblanco-ece/warehouse/gold/top_products-e576865a18e54cdcb82d09ceea44e629/data/`. Trino n'a pas choisi de chemin : à la création de la table, il demande au catalogue l'emplacement du namespace `gold`. Polaris le dérive de la `default-base-location` du catalogue (s3://<bucket>/warehouse/) suivie de gold/. Trino y ajoute le nom de la table, suffixé d'un identifiant unique (option iceberg.unique-table-location, activée par défaut). Polaris vérifie que cet emplacement est dans les « allowed locations » du catalogue.

Validation

Commande

```
sql("""
SELECT sum(quantity) AS v FROM lakehouse.silver.orders
""")
```

Sortie

```
      v
0  8615
```

Commande

```
sql("""
SELECT sum(units) AS v FROM lakehouse.gold.orders_by_state_month
""")
```

Sortie

```
      v
0  8615
```

Commande

```
sql("""
SELECT sum(units) AS v FROM lakehouse.gold.top_products
""")
```

Sortie

```
      v
0  8615
```

Commande

```
assert silver_units == gold_units
```

Sortie

```
OK : 8615 == 8615
```

Commande

```
assert silver_units == product_units
```

Sortie

```
OK : 8615 == 8615
```

Commande

```
sql("""
SELECT count(*) AS v FROM lakehouse.silver.users
""")
```

Sortie

```
    v
0  50
```

Commande

```
sql("""
SELECT count(*) AS v FROM lakehouse.gold.user_activity
""")
```

Sortie

```
    v
0  50
```

Commande

```
sql("""
SELECT count(*) AS v FROM lakehouse.silver.orders
""")
```

Sortie

```
      v
0  2829
```

Commande

```
sql("""
SELECT sum(orders) AS v FROM lakehouse.gold.user_activity
""")
```

Sortie

```
      v
0  2829
```

Commande

```
assert silver_users == gold_users
```

Sortie

```
OK : 50 == 50
```

Commande

```
assert silver_orders == gold_orders
```

Sortie

```
OK : 2829 == 2829
```

Rafraîchir une table gold

Commande

```
sql("""
DELETE FROM lakehouse.gold.top_products
""")
```

Sortie

```
   rows
0     6
```

Commande

```
sql("""
INSERT INTO lakehouse.gold.top_products
SELECT product, count(*), sum(quantity),
       rank() OVER (ORDER BY sum(quantity) DESC)
FROM lakehouse.silver.orders
GROUP BY product
""")
```

Sortie

```
   rows
0     6
```

Commande

```
sql("""
SELECT snapshot_id, committed_at, operation
FROM lakehouse.gold."top_products$snapshots"
ORDER BY committed_at
""")
```

Sortie

```
           snapshot_id                committed_at operation
0  6115508730732381724 2026-11-09 11:22:43.505+00:00    append
1  8686583634333997744 2026-11-09 11:23:00.806+00:00    delete
2  5807318055631278364 2026-11-09 11:23:03.514+00:00    append
```

Commande

```
sql("""
SELECT count(*) AS products FROM lakehouse.gold.top_products FOR VERSION AS OF 6115508730732381724
""")
```

Sortie

```
   products
0         6
```

Commande

```
sql("""
SELECT count(*) AS products FROM lakehouse.gold.top_products
""")
```

Sortie

```
   products
0         6
```

Question : Combien de snapshots la table a-t-elle après le rafraîchissement, et quelle est l'opération de chacun ?

3 snapshots, dans l'ordre : append, delete, append. Le CREATE TABLE AS SELECT crée le premier snapshot (append des données initiales). Le DELETE commit un snapshot qui retire toutes les lignes : comme il supprime des fichiers entiers, Trino fait une suppression au niveau des métadonnées, sans réécrire de données. L'INSERT commit un snapshot append avec les nouvelles lignes. Chaque instruction est une transaction autonome, donc un snapshot. Le time travel sur le premier snapshot retrouve les 6 produits initiaux.

Question : Entre le DELETE et l'INSERT, un dashboard lit la table. Que voit-il ? Que changerait un MERGE ?

Il voit une table vide : le snapshot courant est celui du DELETE, qui est déjà commité. Le dashboard affiche donc zéro produit jusqu'à la fin de l'INSERT. Un MERGE applique les mises à jour, les insertions et les suppressions dans une seule transaction, donc un seul snapshot : le lecteur voit soit l'ancienne version complète, soit la nouvelle, jamais d'état intermédiaire. Un `CREATE OR REPLACE TABLE ... AS SELECT` obtient le même effet pour un remplacement complet.

Comparaison avec Spark

Question : Que faut-il à chaque moteur pour exécuter la requête top_products, en plus de la requête elle-même ?

Trino : un cluster déjà déployé (une fois pour toutes, par Helm) avec le catalogue `lakehouse` configuré côté serveur (URI Polaris, identifiant du principal, endpoint et clés S3), plus un client léger (`trino` en Python) et les paramètres de connexion. L'utilisateur ne manipule aucun identifiant et n'a pas de job à soumettre. Spark : une SparkSession à créer (démarrage d'une JVM driver), le package iceberg-spark-runtime téléchargé depuis Maven, les clés S3 (S3A, TemporaryAWSCredentialsProvider) et l'identifiant Polaris configurés dans la session par chaque utilisateur, et un job (shell interactif, spark-submit ou SparkApplication sur Kubernetes) dont il faut gérer les ressources et la fin.

Question : Trino ou Spark : dashboard rafraîchi chaque minute sur gold ; transformation nocturne de 10 To de bronze ; data scientist qui explore silver avec une requête ?

Dashboard chaque minute : Trino, car le service est toujours démarré, la latence est faible, la concurrence aussi, et la couche gold est petite. 10 To la nuit : Spark, qui tolère les pannes (relance des tâches et des stages perdus), traite de gros shuffles en débordant sur disque, et peut monter en charge ponctuellement puis libérer les ressources. Exploration par un data scientist : Trino pour des requêtes SQL rapides et itératives. Spark devient pertinent si l'exploration continue en Python (DataFrame, pandas, ML) sur de gros volumes.

Question : Trino échoue sur toute la requête si un worker est perdu (sans fault-tolerant execution). Pourquoi est-ce acceptable ici, et pas pour 10 To ?

Les requêtes de ce lab durent quelques secondes sur quelques milliers de lignes : les relancer en cas d'échec ne coûte presque rien, et la probabilité de perdre un nœud pendant ce temps est négligeable. Une transformation de 10 To dure des heures sur de nombreux nœuds. La probabilité d'une panne croît avec la durée et le nombre de nœuds, et tout recommencer coûte des heures de calcul. Il faut alors pouvoir relancer seulement les tâches perdues : c'est ce que fait Spark grâce à son lignage, et ce que fait Trino avec retry-policy=TASK et un exchange manager qui écrit les résultats intermédiaires sur un stockage externe.

Lab 3 : Star schema

Même connexion Python que le lab 2 (la session reste ouverte).

Dimensions

Commande

```
sql("""
CREATE TABLE lakehouse.gold.dim_user AS
SELECT
    row_number() OVER (ORDER BY user_id) AS user_key,
    user_id, username, name, sex, birthdate, city, state, zip_code
FROM lakehouse.silver.users
""")
```

Sortie

```
   rows
0    50
```

Commande

```
sql("""
CREATE TABLE lakehouse.gold.dim_product AS
SELECT row_number() OVER (ORDER BY product) AS product_key, product
FROM (SELECT DISTINCT product FROM lakehouse.silver.orders)
""")
```

Sortie

```
   rows
0     6
```

Commande

```
sql("""
SELECT CAST(min(ordered_at) AS date) AS first_day,
       CAST(max(ordered_at) AS date) AS last_day
FROM lakehouse.silver.orders
""")
```

Sortie

```
    first_day    last_day
0  2020-01-01  2020-04-27
```

Commande

```
sql("""
CREATE TABLE lakehouse.gold.dim_date AS
SELECT
    CAST(date_format(CAST(d AS timestamp), '%Y%m%d') AS integer) AS date_key,
    d                                    AS full_date,
    year(d)                              AS year,
    quarter(d)                           AS quarter,
    month(d)                             AS month,
    date_format(CAST(d AS timestamp), '%M') AS month_name,
    day_of_week(d)                       AS day_of_week,
    day_of_week(d) IN (6, 7)             AS is_weekend
FROM UNNEST(sequence(DATE '2020-01-01', DATE '2020-04-27', INTERVAL '1' DAY)) AS t(d)
""")
```

Sortie

```
   rows
0   118
```

Table de faits

Commande

```
sql("""
CREATE TABLE lakehouse.gold.fact_orders AS
SELECT
    o.order_id,
    u.user_key,
    p.product_key,
    CAST(date_format(o.ordered_at, '%Y%m%d') AS integer) AS date_key,
    o.ordered_at,
    o.quantity
FROM lakehouse.silver.orders AS o
LEFT JOIN lakehouse.gold.dim_user    AS u ON u.user_id = o.user_id
LEFT JOIN lakehouse.gold.dim_product AS p ON p.product = o.product
""")
```

Sortie

```
   rows
0  2829
```

Commande

```
sql("""
SELECT 'dim_user' AS table_name, count(*) AS row_count FROM lakehouse.gold.dim_user
UNION ALL SELECT 'dim_product', count(*) FROM lakehouse.gold.dim_product
UNION ALL SELECT 'dim_date', count(*) FROM lakehouse.gold.dim_date
UNION ALL SELECT 'fact_orders', count(*) FROM lakehouse.gold.fact_orders
""")
```

Sortie

```
    table_name  row_count
0     dim_user         50
1  dim_product          6
2  fact_orders       2829
3     dim_date        118
```

Le lab annonce 5000 lignes dans fact_orders. Avec le jeu de données réellement généré, on attend autant de lignes que silver.orders (2829).

Interroger l'étoile

Commande

```
sql("""
SELECT d.year, u.state, p.product,
       count(*)        AS orders,
       sum(f.quantity) AS units
FROM lakehouse.gold.fact_orders AS f
JOIN lakehouse.gold.dim_date    AS d ON d.date_key    = f.date_key
JOIN lakehouse.gold.dim_user    AS u ON u.user_key    = f.user_key
JOIN lakehouse.gold.dim_product AS p ON p.product_key = f.product_key
GROUP BY d.year, u.state, p.product
ORDER BY units DESC
LIMIT 10
""")
```

Sortie

```
   year state    product  orders  units
0  2020    FM    brioche      43    134
1  2020    VI      bread      37    110
2  2020    HI      bread      35    110
3  2020    HI  croissant      36    107
4  2020    VI  croissant      34    107
5  2020    VI    brioche      33    104
6  2020    FM  croissant      32    102
7  2020    AE      bread      31    100
8  2020    FM      donut      33     98
9  2020    VT      drink      33     96
```

Commande

```
sql("""
SELECT u.state, d.year, d.month,
       count(*)                  AS orders,
       sum(f.quantity)           AS units,
       count(DISTINCT f.user_key) AS buyers
FROM lakehouse.gold.fact_orders AS f
JOIN lakehouse.gold.dim_date    AS d ON d.date_key = f.date_key
JOIN lakehouse.gold.dim_user    AS u ON u.user_key = f.user_key
GROUP BY u.state, d.year, d.month
ORDER BY d.year, d.month, u.state
LIMIT 10
""")
```

Sortie

```
  state  year  month  orders  units  buyers
0    AL  2020      1      65    199       1
1    AP  2020      1      17     49       1
2    AS  2020      1      12     40       1
3    CO  2020      1      17     43       1
4    FM  2020      1      66    200       1
5    GA  2020      1       4     12       1
6    IN  2020      1      49    175       1
7    KY  2020      1      83    256       1
8    MH  2020      1      21     57       1
9    NE  2020      1      11     30       1
```

Contrôle exhaustif (au-delà des 10 premières lignes) : différence symétrique entre l'agrégat gold et sa reconstruction depuis l'étoile.

Commande

```
sql("""
WITH agg AS (
    SELECT state, year(month) AS year, month(month) AS month, orders, units, buyers
    FROM lakehouse.gold.orders_by_state_month
), star AS (
    SELECT u.state, d.year, d.month, count(*) AS orders, sum(f.quantity) AS units,
           count(DISTINCT f.user_key) AS buyers
    FROM lakehouse.gold.fact_orders AS f
    JOIN lakehouse.gold.dim_date AS d ON d.date_key = f.date_key
    JOIN lakehouse.gold.dim_user AS u ON u.user_key = f.user_key
    GROUP BY u.state, d.year, d.month
)
SELECT (SELECT count(*) FROM (SELECT * FROM agg EXCEPT SELECT * FROM star)) AS only_in_agg,
       (SELECT count(*) FROM (SELECT * FROM star EXCEPT SELECT * FROM agg)) AS only_in_star
""")
```

Sortie

```
   only_in_agg  only_in_star
0            0             0
```

Validation

Commande

```
sql("""
SELECT count(*) AS v FROM lakehouse.silver.orders
""")
```

Sortie

```
      v
0  2829
```

Commande

```
sql("""
SELECT count(*) AS v FROM lakehouse.gold.fact_orders
""")
```

Sortie

```
      v
0  2829
```

Commande

```
sql("""
SELECT sum(quantity) AS v FROM lakehouse.silver.orders
""")
```

Sortie

```
      v
0  8615
```

Commande

```
sql("""
SELECT sum(quantity) AS v FROM lakehouse.gold.fact_orders
""")
```

Sortie

```
      v
0  8615
```

Commande

```
assert silver_orders == fact_orders
```

Sortie

```
OK : 2829 == 2829
```

Commande

```
assert silver_units == fact_units
```

Sortie

```
OK : 8615 == 8615
```

Commande

```
sql("""
SELECT count(*) AS v
FROM lakehouse.gold.fact_orders AS f
LEFT JOIN lakehouse.gold.dim_user    AS u ON u.user_key    = f.user_key
LEFT JOIN lakehouse.gold.dim_product AS p ON p.product_key = f.product_key
LEFT JOIN lakehouse.gold.dim_date    AS d ON d.date_key    = f.date_key
WHERE u.user_key IS NULL OR p.product_key IS NULL OR d.date_key IS NULL
""")
```

Sortie

```
   v
0  0
```

Commande

```
assert unmatched == 0
```

Sortie

```
OK : 0 == 0
```

Contrôles supplémentaires (réponse à la question 1)

Commande

```
sql("""
SELECT count(*) AS mismatches
FROM lakehouse.gold.fact_orders AS f
JOIN lakehouse.silver.orders AS o ON o.order_id = f.order_id
LEFT JOIN lakehouse.gold.dim_user    AS u ON u.user_key    = f.user_key
LEFT JOIN lakehouse.gold.dim_product AS p ON p.product_key = f.product_key
LEFT JOIN lakehouse.gold.dim_date    AS d ON d.date_key    = f.date_key
WHERE u.user_id   IS DISTINCT FROM o.user_id
   OR p.product   IS DISTINCT FROM o.product
   OR d.full_date IS DISTINCT FROM CAST(o.ordered_at AS date)
   OR f.quantity  IS DISTINCT FROM o.quantity
""")
```

Sortie

```
   mismatches
0           0
```

Commande

```
sql("""
SELECT
  (SELECT count(*) - count(DISTINCT order_id) FROM lakehouse.gold.fact_orders) AS dup_order_id,
  (SELECT count(*) - count(DISTINCT user_id)  FROM lakehouse.gold.dim_user)    AS dup_user_id,
  (SELECT count(*) - count(DISTINCT product)  FROM lakehouse.gold.dim_product) AS dup_product
""")
```

Sortie

```
   dup_order_id  dup_user_id  dup_product
0             0            0            0
```

Question : Ces contrôles suffisent-ils à garantir que chaque commande est correctement représentée ? Exemple de défaut non détecté, et contrôles à ajouter.

Non. Les trois assertions vérifient des totaux (nombre de lignes, somme des quantités) et la présence d'une clé, pas la justesse de chaque ligne. Un défaut qui conserve les totaux passe inaperçu : par exemple deux commandes dont les user_key sont intervertis, un product_key qui pointe vers le mauvais produit (clés décalées après une reconstruction de dim_product sans reconstruire les faits), un date_key calculé dans le mauvais fuseau qui place une commande proche de minuit sur la veille, ou une commande dupliquée compensée par une commande perdue de même quantité. Contrôles ajoutés et exécutés ci-dessus : (1) réconciliation ligne à ligne par order_id entre la table de faits et silver, en comparant user_id, product, date et quantity via les dimensions : 0 écart(s) ; (2) unicité de order_id dans les faits et des clés naturelles dans les dimensions : doublons = 0 / 0 / 0. On peut aussi comparer des agrégats par dimension (unités par produit, par état, par jour) entre silver et l'étoile.

Question : Quel est le grain de fact_orders ? De dim_date ?

fact_orders : une ligne par commande (order_id), soit le grain de silver.orders. dim_date : une ligne par jour calendaire entre la première et la dernière commande, identifiée par date_key = AAAAMMJJ.

Question : Quelle assertion échoue si silver.users contient deux fois le même user_id ? Laquelle si une commande n'a pas de produit (NULL) ?

user_id en double : dim_user contient deux lignes, avec deux user_key, pour le même user_id. Le LEFT JOIN sur user_id duplique alors toutes les commandes de cet utilisateur : fact_orders a plus de lignes que silver.orders, et `silver_orders == fact_orders` échoue (ainsi que `silver_units == fact_units`). Produit NULL : SELECT DISTINCT garde le NULL, donc dim_product reçoit une ligne avec product NULL. Mais l'égalité `p.product = o.product` n'est jamais vraie avec NULL : la commande garde product_key NULL. Les comptes et les sommes restent justes ; c'est l'assertion `unmatched == 0` qui échoue.

Question : Clés de substitution générées par row_number() : que devient user_key si dim_user est reconstruite après l'ajout d'un utilisateur ? Conséquence pour fact_orders ?

row_number() numérote par ordre de user_id. Un nouvel utilisateur dont l'UUID se place au milieu décale de 1 la clé de tous les utilisateurs suivants. Les clés ne sont donc pas stables d'une reconstruction à l'autre. Si fact_orders n'est pas reconstruite en même temps, ses user_key désignent les mauvais utilisateurs, sans aucune erreur, et l'assertion `unmatched` ne le voit pas. Il faut soit toujours reconstruire les faits avec les dimensions, soit générer des clés stables : attribuer max(user_key) + n aux seuls nouveaux utilisateurs avec un MERGE, ou dériver la clé d'un hash de la clé naturelle.

Question : Si un utilisateur déménage, quel état montrent ses commandes passées ? Que faudrait-il pour garder l'historique ?

dim_user écrase les attributs (Slowly Changing Dimension de type 1) : toutes les commandes, y compris les anciennes, affichent le nouvel état, et l'historique par état est réécrit. Pour garder l'historique, il faut une SCD de type 2 : une ligne par version de l'utilisateur, avec sa propre user_key, des colonnes valid_from, valid_to et is_current. La table de faits rattache chaque commande à la version valide à la date de commande (user_id = ... AND ordered_at >= valid_from AND ordered_at < valid_to).

Question : Comparer orders_by_state_month et la requête en étoile qui la reconstruit. Quelles questions chacune traite-t-elle sans nouvelle table, et que coûte l'étoile ?

Les deux donnent les mêmes chiffres (différence symétrique : only_in_agg = 0, only_in_star = 0). L'agrégat ne répond qu'à sa question (état x mois), mais il est petit, précalculé et lu instantanément par un outil BI. L'étoile répond à toute combinaison de dimensions sans nouvelle table : par produit, année, trimestre, week-end, ville, âge, etc. En contrepartie, chaque requête paie à l'exécution les jointures entre la table de faits et les dimensions, puis l'agrégation sur toutes les lignes de faits. C'est négligeable ici, mais cela croît avec le volume. Elle demande aussi de connaître le modèle pour écrire les jointures. En pratique, on combine les deux : l'étoile comme socle, et des agrégats pour les tableaux de bord les plus consultés.

Lab 3 : Exercices

Exercice 1 : table partitionnée par mois

Commande

```
sql("""
CREATE TABLE lakehouse.gold.orders_by_state_month_part
WITH (partitioning = ARRAY['month(month)']) AS
SELECT * FROM lakehouse.gold.orders_by_state_month
""")
```

Sortie

```
   rows
0    50
```

Commande

```
sql("""
SELECT partition, file_path, record_count
FROM lakehouse.gold."orders_by_state_month_part$files"
""")
```

Sortie

```
           partition                                                
```

Sortie

```
Trino version: 480
Queued: 636.00us, Analysis: 130.91ms, Planning: 45.58ms, Execution: 552.68ms, Finishing: 0.00ns
Fragment 1 [SINGLE]
    CPU: 2.37ms, Scheduled: 27.50ms, Blocked 899.50ms (Input: 598.42ms, Output: 0.00ns), Input: 1 row (9B); per task:
avg.: 1.00 std.dev.: 0.00, Output: 1 row (9B)
    Peak Memory: 96B, Tasks count: 1; per task: max: 96B
    Output layout: [sum]
    Output partitioning: SINGLE []
    Aggregate[type = FINAL]
    │   Layout: [sum:bigint]
    │   Estimates: {rows: 1 (9B), cpu: 113, memory: 9B, network: 0B}
    │   CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 0.00ns (0.00%), Output: 1 row (9B)
    │   Input avg.: 1.00 rows, Input std.dev.: 0.00%
    │   sum := sum(sum_0)
    └─ LocalExchange[partitioning = SINGLE]
       │   Layout: [sum_0:bigint]
       │   Estimates: {rows: 13 (113B), cpu: 0, memory: 0B, network: 0B}
       │   CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 301.00ms (33.48%), Output: 1 row (9B)
       │   Input avg.: 0.50 rows, Input std.dev.: 100.00%
       └─ RemoteSource[sourceFragmentIds = [2]]
              Layout: [sum_0:bigint]
              CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 598.00ms (66.52%), Output: 1 row (9B)
              Input avg.: 0.50 rows, Input std.dev.: 100.00%

Fragment 2 [SOURCE]
    CPU: 8.95ms, Scheduled: 29.93ms, Blocked 0.00ns (Input: 0.00ns, Output: 0.00ns), Input: 50 rows (1.03kB); per
task: avg.: 50.00 std.dev.: 0.00, Output: 1 row (9B)
    Peak Memory: 96B, Tasks count: 1; per task: max: 96B
    Output layout: [sum_0]
    Output partitioning: SINGLE []
    Aggregate[type = PARTIAL]
    │   Layout: [sum_0:bigint]
    │   Estimates: {rows: 13 (113B), cpu: ?, memory: ?, network: ?}
    │   CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 0.00ns (0.00%), Output: 1 row (9B)
    │   Input avg.: 17.00 rows, Input std.dev.: 0.00%
    │   sum_0 := sum(units)
    └─ ScanFilterProject[table = lakehouse:gold.orders_by_state_month$data@1671412390064957480, filterPredicate = (
month = timestamp(6) with time zone '2020-01-01 00:00:00.000000 UTC')]
           Layout: [units:bigint]
           Estimates: {rows: 50 (450B), cpu: 1.07k, memory: 0B, network: 0B}/{rows: 13 (113B), cpu: 1.07k, memory: 0B,
 network: 0B}/{rows: 13 (113B), cpu: 113, memory: 0B, network: 0B}
           CPU: 8.00ms (100.00%), Scheduled: 29.00ms (100.00%), Blocked: 0.00ns (0.00%), Output: 17 rows (153B)
           Input avg.: 50.00 rows, Input std.dev.: 0.00%
           units := 4:units:bigint
... (3 lignes supplémentaires coupées)
```

Commande

```
print(sql("""
EXPLAIN ANALYZE SELECT sum(units) FROM lakehouse.gold.orders_by_state_month_part WHERE month = TIMESTAMP '2020-01-01
00:00:00 UTC'
""").iloc[0, 0])
```

Sortie

```
Trino version: 480
Queued: 377.86us, Analysis: 203.44ms, Planning: 197.37ms, Execution: 612.35ms, Finishing: 0.00ns
Fragment 1 [SINGLE]
    CPU: 1.79ms, Scheduled: 1.83ms, Blocked 729.76ms (Input: 519.06ms, Output: 0.00ns), Input: 1 row (9B); per task:
avg.: 1.00 std.dev.: 0.00, Output: 1 row (9B)
    Peak Memory: 96B, Tasks count: 1; per task: max: 96B
    Output layout: [sum]
    Output partitioning: SINGLE []
    Aggregate[type = FINAL]
    │   Layout: [sum:bigint]
    │   Estimates: {rows: 1 (9B), cpu: 153, memory: 9B, network: 0B}
    │   CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 0.00ns (0.00%), Output: 1 row (9B)
    │   Input avg.: 1.00 rows, Input std.dev.: 0.00%
    │   sum := sum(sum_0)
    └─ LocalExchange[partitioning = SINGLE]
       │   Layout: [sum_0:bigint]
       │   Estimates: {rows: 17 (153B), cpu: 0, memory: 0B, network: 0B}
       │   CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 211.00ms (28.90%), Output: 1 row (9B)
       │   Input avg.: 0.50 rows, Input std.dev.: 100.00%
       └─ RemoteSource[sourceFragmentIds = [2]]
              Layout: [sum_0:bigint]
              CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 519.00ms (71.10%), Output: 1 row (9B)
              Input avg.: 0.50 rows, Input std.dev.: 100.00%

Fragment 2 [SOURCE]
    CPU: 13.37ms, Scheduled: 54.19ms, Blocked 0.00ns (Input: 0.00ns, Output: 0.00ns), Input: 17 rows (374B); per task:
 avg.: 17.00 std.dev.: 0.00, Output: 1 row (9B)
    Peak Memory: 96B, Tasks count: 1; per task: max: 96B
    Output layout: [sum_0]
    Output partitioning: SINGLE []
    Aggregate[type = PARTIAL]
    │   Layout: [sum_0:bigint]
    │   Estimates: {rows: 17 (153B), cpu: ?, memory: ?, network: ?}
    │   CPU: 0.00ns (0.00%), Scheduled: 2.00ms (3.70%), Blocked: 0.00ns (0.00%), Output: 1 row (9B)
    │   Input avg.: 17.00 rows, Input std.dev.: 0.00%
    │   sum_0 := sum(units)
    └─ ScanFilterProject[table = lakehouse:gold.orders_by_state_month_part$data@2690734992821581558, filterPredicate
= (month = timestamp(6) with time zone '2020-01-01 00:00:00.000000 UTC')]
           Layout: [units:bigint]
           Estimates: {rows: 17 (153B), cpu: 374, memory: 0B, network: 0B}/{rows: 17 (153B), cpu: 374, memory: 0B,
network: 0B}/{rows: 17 (153B), cpu: 153, memory: 0B, network: 0B}
           CPU: 13.00ms (100.00%), Scheduled: 52.00ms (96.30%), Blocked: 0.00ns (0.00%), Output: 17 rows (153B)
           Input avg.: 17.00 rows, Input std.dev.: 0.00%
           units := 4:units:bigint
... (3 lignes supplémentaires coupées)
```

Résultat : la table non partitionnée a 1 fichier(s), la table partitionnée 4 (au moins un par mois). Lignes lues par le scan pour le mois 2020-01 : 50 sans partitionnement, contre 17 avec. Le filtre sur `month` correspond à la transformation de partition month(month) : Trino élague les fichiers des autres mois à partir des statistiques de partition des manifestes, avant de lire les données.

Exercice 2 : cohortes par décennie de naissance

Commande

```
sql("""
CREATE TABLE lakehouse.gold.cohorts AS
SELECT
    (year(u.birthdate) / 10) * 10      AS birth_decade,
    count(DISTINCT u.user_id)          AS users,
    count(o.order_id)                  AS orders,
    round(avg(o.quantity), 2)          AS avg_units_per_order
FROM lakehouse.silver.users AS u
LEFT JOIN lakehouse.silver.orders AS o ON o.user_id = u.user_id
GROUP BY 1
""")
```

Sortie

```
   rows
0    11
```

Commande

```
sql("""
SELECT * FROM lakehouse.gold.cohorts ORDER BY birth_decade
""")
```

Sortie

```
    birth_decade  users  orders  avg_units_per_order
0           1910      1      47                 3.19
1           1930      5     198                 3.14
2           1940      4     273                 3.06
3           1950      6     340                 3.01
4           1960      8     433                 3.02
5           1970      5     369                 2.99
6           1980      2      81                 2.79
7           1990      5     402                 3.07
8           2000      4     154                 2.99
9           2010      5     244                 3.08
10          2020      5     288                 3.12
```

count(DISTINCT user_id) est nécessaire, car la jointure multiplie chaque utilisateur par son nombre de commandes. avg(quantity) ignore les NULL des utilisateurs sans commande et donne directement la moyenne d'unités par commande.

Exercice 4 : OPTIMIZE

Exécuté avant l'exercice 3, qui modifie silver.

Commande

```
sql("""
SELECT snapshot_id, committed_at, operation FROM lakehouse.gold."top_products$snapshots" ORDER BY committed_at
""")
```

Sortie

```
           snapshot_id                  committed_at operation
0  6115508730732381724 2026-11-09 11:22:43.505+00:00    append
1  8686583634333997744 2026-11-09 11:23:00.806+00:00    delete
2  5807318055631278364 2026-11-09 11:23:03.514+00:00    append
```

Commande

```
sql("""
SELECT file_path, record_count, file_size_in_bytes FROM lakehouse.gold."top_products$files"
""")
```

Sortie

```
                                           file_path  record_count  file_size_in_bytes
0  s3://user-g-zinck--diasblanco-ece/warehouse/gold/top_products-e576865a18e54cdcb82d09ceea44e629/data/
20261109_112301_00058_7c7wa-c9fb4562-b3d6-47d1-bab2-a17b92f632e8.parquet             6                 760
```

Commande

```
sql("""
ALTER TABLE lakehouse.gold.top_products EXECUTE optimize
""")
```

Sortie

```
(0 ligne)
```

Commande

```
sql("""
SELECT snapshot_id, committed_at, operation FROM lakehouse.gold."top_products$snapshots" ORDER BY committed_at
""")
```

Sortie

```
           snapshot_id                  committed_at operation
0  6115508730732381724 2026-11-09 11:22:43.505+00:00    append
1  8686583634333997744 2026-11-09 11:23:00.806+00:00    delete
2  5807318055631278364 2026-11-09 11:23:03.514+00:00    append
```

Commande

```
sql("""
SELECT file_path, record_count, file_size_in_bytes FROM lakehouse.gold."top_products$files"
""")
```

Sortie

```
                                                file_path  record_count  file_size_in_bytes
0  s3://user-g-zinck--diasblanco-ece/warehouse/gold/top_products-e576865a18e54cdcb82d09ceea44e629/data/
20261109_112301_00058_7c7wa-c9fb4562-b3d6-47d1-bab2-a17b92f632e8.parquet             6                 760
```

Avant : 3 snapshots, 1 fichier(s) dans le snapshot courant. Après : 3 snapshots, 1 fichier(s). OPTIMIZE réécrit les petits fichiers (et applique les fichiers de suppression éventuels) en fichiers plus gros, puis commit un nouveau snapshot d'opération « replace » : les données logiques ne changent pas, seule leur organisation physique change. Si la table n'a qu'un seul petit fichier, rien n'est réécrit et aucun snapshot n'est ajouté. Les anciens fichiers restent dans le bucket tant que les snapshots qui les référencent ne sont pas expirés (expire_snapshots puis remove_orphan_files).

Exercice 3 : INSERT dans silver, MERGE dans gold, time travel

Commande

```
sql("""
SELECT user_id, state FROM lakehouse.silver.users ORDER BY user_id LIMIT 1
""")
```

Sortie

```
                               user_id state
0  09a0a0f6-73b0-4091-b634-c1694f76e388    VT
```

Commande

```
sql("""
SELECT snapshot_id FROM lakehouse.gold."orders_by_state_month$snapshots"
ORDER BY committed_at DESC LIMIT 1
""")
```

Sortie

```
           snapshot_id
0  1671412390064957480
```

Commande

```
sql("""
INSERT INTO lakehouse.silver.orders (order_id, user_id, ordered_at, quantity, product)
VALUES ('00000000-0000-4000-8000-000000000001', '09a0a0f6-73b0-4091-b634-c1694f76e388', TIMESTAMP '2020-01-15 12:00:
00.000000 UTC', 3, 'croissant')
""")
```

Sortie

```
   rows
0     1
```

Commande

```
sql("""
MERGE INTO lakehouse.gold.orders_by_state_month AS t
USING (
    SELECT u.state, date_trunc('month', o.ordered_at) AS month,
           count(*) AS orders, sum(o.quantity) AS units,
           count(DISTINCT o.user_id) AS buyers
    FROM lakehouse.silver.orders AS o
    JOIN lakehouse.silver.users  AS u ON u.user_id = o.user_id
    GROUP BY 1, 2
) AS s
ON t.state = s.state AND t.month = s.month
WHEN MATCHED AND (t.orders <> s.orders OR t.units <> s.units OR t.buyers <> s.buyers)
    THEN UPDATE SET orders = s.orders, units = s.units, buyers = s.buyers
WHEN NOT MATCHED
    THEN INSERT (state, month, orders, units, buyers)
         VALUES (s.state, s.month, s.orders, s.units, s.buyers)
""")
```

Sortie

```
   rows
0     1
```

Commande

```
sql("""
SELECT * FROM lakehouse.gold.orders_by_state_month FOR VERSION AS OF 1671412390064957480 WHERE state = 'VT' AND month
= TIMESTAMP '2020-01-01 00:00:00 UTC'
""")
```

Sortie

```
(0 ligne)
```

Commande

```
sql("""
SELECT * FROM lakehouse.gold.orders_by_state_month WHERE state = 'VT' AND month = TIMESTAMP '2020-01-01 00:00:00 UTC'
""")
```

Sortie

```
  state                     month  orders  units  buyers
0    VT 2020-01-01 00:00:00+00:00       1      3       1
```

Commande

```
sql("""
SELECT snapshot_id, committed_at, operation FROM lakehouse.gold."orders_by_state_month$snapshots" ORDER BY
committed_at
""")
```

Sortie

```
           snapshot_id                  committed_at  operation
0  1671412390064957480 2026-11-09 11:22:37.708+00:00     append
1  8098786819618149309 2026-11-09 11:24:07.960+00:00  overwrite
```

La commande ajoutée pour l'utilisateur de l'état VT en janvier 2020 augmente orders de 1 et units de 3 sur une seule ligne. Le MERGE ne met à jour que les lignes dont les agrégats ont changé, en un seul snapshot, donc de façon atomique pour les lecteurs. `FOR VERSION AS OF` relit la version précédente à partir de son snapshot_id. Remarque : la commande reste dans silver.orders ; ce script la supprime au début de sa prochaine exécution.

Exercice 5 : même agrégation depuis Spark

Non exécuté par ce script, car il demande une session Spark connectée à Polaris (cf. lab Polaris). Dans cette session : spark.sql("SELECT u.state, date_trunc('month', o.ordered_at) m, count(*) FROM polaris.silver.orders o JOIN polaris.silver.users u USING (user_id) GROUP BY 1, 2").count() doit donner le même nombre de lignes que gold.orders_by_state_month, car les deux moteurs lisent les mêmes fichiers via le même catalogue.